

Презентация по лабораторной работе номер 7

Основы информационной безопасности

Татьяна Александровна Пономарева

2026-04-15

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Выполнение лабораторной работы
- 4 Выводы

Раздел 1

Информация

- Пономарева Татьяна Александровна



- Пономарева Татьяна Александровна
- Студент группы НКАбд-04-24



- Пономарева Татьяна Александровна
- Студент группы НКАбд-04-24
- Российский университет дружбы народов им.
П. Лумумбы



- Пономарева Татьяна Александровна
- Студент группы НКАбд-04-24
- Российский университет дружбы народов им.
П. Лумумбы
- taronomareva6742@gmail.com



- Пономарева Татьяна Александровна
- Студент группы НКАбд-04-24
- Российский университет дружбы народов им. П. Лумумбы
- taonomareva6742@gmail.com
- <https://github.com/taonomareva>



Раздел 2

Вводная часть

Цель работы

Освоить на практике применение режима однократного гаммирования

Разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного кодирования

Гаммирование – это наложение (снятие) на открытые (зашифрованные) данные криптографической гаммы, то есть последовательности элементов данных, вырабатываемых с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных.

С точки зрения теории криптоанализа, метод шифрования случайной однократной равновероятной гаммой той же длины, что и открытый текст, является невскрываемым. Кроме того, даже раскрыв часть сообщения, дешифровщик не сможет хоть сколько-нибудь поправить положение – информация о вскрытом участке гаммы не дает информации об остальных ее частях. «Наложение» гаммы не что иное, как выполнение операции сложения по модулю 2 [1].

Раздел 3

Выполнение лабораторной работы

Создание файла lab7.py и его содержимое

Создаем файл lab7.py и пишем код на языке Python для выполнения задания (рис. 1).

```
laponomarev@laponomareva:~$ nano lab7.py
GNU nano 8.1 lab7.py
def xor_bytes(data_bytes, key_bytes):
    # Операция XOR между каждым байтом данных и ключа
    return bytes([b ^ k for b, k in zip(data_bytes, key_bytes)])

def task_1():
    print("---- Задание 1: Шифрование ----")
    plaintext = "С Новым Годом, друзья!"
    # Кодировем текст в байты (используем cp1251 или utf-8)
    p_bytes = plaintext.encode('utf-8')

    # Генерируем произвольный ключ той же длины (например, из случайных чисел или фразы)
    # Для простоты возьмем фразу "ЭтоСекретныйКлюч12345" нужной длины
    key_phrase = "ЭтоСекретныйКлюч1234567"
    k_bytes = key_phrase.encode('utf-8')[:len(p_bytes)]

    # Шифруем C = P XOR K
    c_bytes = xor_bytes(p_bytes, k_bytes)

    print(f"Оригинальный текст: {plaintext}")
    print(f"Ключ (HEX): {k_bytes.hex().upper()}")
    print(f"Шифротекст (HEX): {c_bytes.hex().upper()}")
    return c_bytes, k_bytes

def task_2(cipher_hex):
    print("\n---- Задание 2: Поиск альтернативного ключа ----")
    c_bytes = bytes.fromhex(cipher_hex)

    # Мы хотим, чтобы тот же шифротекст расшифровался как "Привет от Мэллери!"
    fake_text = "Привет от Мэллери!"
    f_bytes = fake_text.encode('utf-8')[:len(c_bytes)]

    # Находим фальшивый ключ: K_fake = C XOR P_fake
    k_fake = xor_bytes(c_bytes[:len(f_bytes)], f_bytes)

    print(f"Желаемый ложный текст: {fake_text}")
    print(f"Необходимый для этого ключ (HEX): {k_fake.hex().upper()}")

    # Проверка
    decrypted = xor_bytes(c_bytes[:len(f_bytes)], k_fake).decode('utf-8')
    print(f"Проверка расшифровки: {decrypted}")

# Запуск
cipher_result, key_result = task_1()
task_2(cipher_result.hex())
```

Рисунок 1: Создание файла lab7.py и его содержимое

Запускаем файл lab7.py (рис. 2).

```
taponomareva@taponomareva:~$ python lab7.py
--- Задание 1: Шифрование ---
Открытый текст: С Новым Годом, друзья!
Ключ (HEX):      D0ADD182D0BED0A1D0B5D0BAD180D0B5D182D0BDD18BD0B9D09AD08BD18ED18721313233343536
Шифротекст (HEX): 000CF1524D6E6E7162645B6A6DA00026013C000901350005FCBA000F000E0004F186E3BFE5BA17

--- Задание 2: Поиск альтернативного ключа ---
Желаемый ложный текст: Привет от Моллера!!!
Необходимый для этого ключ (HEX): D09320D29DD6BEC3B2D18AE84D70BEF7831CD095D08BD0BE2C01D0BAD18ED0B4D0A7C2
Проверка расшифровки: Привет от Моллера!!!
```

Рисунок 2: Запуск lab7.py

Раздел 4

Выводы

В ходе выполнения лабораторной работы применение режима однократного гаммирования было освоено на практике

Раздел 5

Список литературы

- 1 StudFiles. Режим шифрования однократного гаммирования: Программные средства защиты информации [Электронный ресурс]. НГТУ (Новосибирский государственный технический университет). URL: <https://studfile.net/preview/11550163/page:15/> (дата обращения: 15.04.2026).